Guía Docente y Solucionario Comentado

Herramienta Pedagógica para el Acompañamiento en Clases Particulares

Cátedra de Paradigmas y Lenguajes de Programación — Material de Soporte Docente

> 📌 **Orientación Didáctica para el Ayudante**
>
> Los alumnos que desaprueban suelen tener una **parálisis por sobrecarga sintáctica**: confunden los paréntesis con la lógica. La clave para destrabarla es hacer que **verbalice** en español qué tiene que pasar antes de escribir cualquier paréntesis.

MÓDULO 1: Anatomía de Listas y Cirugía de Accesores (Soluciones)

Ejercicio 1.1 — Soluciones V/F

VERDADERO **1.** `(cdr '(42))` devuelve `nil`. Una lista de un elemento tiene un cons cuyo cdr apunta a la lista vacía.

FALSO **2.** `(cons '(1 2) '(3 4))` da `((1 2) 3 4)`. **Alerta:** La alumna suele confundir cons con append. Remarcarle que cons siempre agrega un único elemento entero al principio.

FALSO **3.** Extrae `C`, no `B`. `(cadr l)` es `(C D)`, y su car es `C`. Para `B` se usa `(cadar l)`.

FALSO **4.** `append` da una lista plana de 4 elementos `(A B C D)`; `list` da una lista de 2 sublistas `((A B) (C D))`.

VERDADERO **5.** En LISP estándar y XLISP `(car nil)` y `(cdr nil)` dan `nil` sin romperse.

Ejercicio 1.2 — Retornos de M

Lista: `'(CAFE (BRASIL MEDIO) 1500 (LECHE AZUCAR))`

- `(car M)` → **CAFE**
- `(cdr M)` → **((BRASIL MEDIO) 1500 (LECHE AZUCAR))**
- `(cadr M)` → **(BRASIL MEDIO)**
- `(caddr M)` → **1500**
- `(car (cadr M))` → **BRASIL**
- `(cadr (cadr M))` → **MEDIO**
- `(cadddr M)` → **(LECHE AZUCAR)**
- `(car (cadddr M))` → **LECHE**

Ejercicio 1.3 y 1.4 — Accesores y Caza de Bichos

```
(defun obtener-mesa (p)  (car p))
(defun obtener-mozo (p)  (car (cadr p)))
(defun obtener-turno (p) (cadr (cadr p)))
(defun obtener-total (p) (cadddr p))
(defun obtener-estado (p) (car (cddddr p)))  ;; o (nth 4 p)
```

Sobre el caso Susana: Susana escribía un accesor distinto para cada posición porque no entendía la generalización funcional. Explicarle que en Lisp nunca sabemos si la lista tiene 2, 5 o 1000 elementos: la función siempre hace `(car l)` sobre el primero y avanza con `(cdr l)`.

MÓDULO 2: Lógica Condicional y Predicados Puros (Soluciones)

Ejercicio 2.1 — Soluciones V/F

FALSO **1.** `=` solo funciona con números. Con símbolos arroja error de tipo. Debe usarse `equal`.

FALSO **2.** No arroja error: devuelve `nil` silenciosamente al no cumplirse ninguna condición.

VERDADERO **3.** `(T valor)` actúa como la rama default (else).

FALSO **4.** El `0` es verdadero (*truthy*) en LISP. Lo único falso es `nil`.

FALSO **5.** `and` devuelve el valor del último operando, en este caso el átomo `'DESCUENTO`, no `T`.

Ejercicio 2.2 y 2.4 — Retornos y Error de Valen

- `(clasificar-pedido 3500 4)` → **PREMIUM-EXPRESS**
- `(clasificar-pedido 3200 12)` → **PREMIUM**
- `(clasificar-pedido 1500 3)` → **EXPRESS**
- `(clasificar-pedido 2000 10)` → **ESTANDAR**
- `(clasificar-pedido 'MUCHO 5)` → **DATOS-INVALIDOS**

Error de Valen: Puso `((equal estado 'LISTO))` sin resultado. Como la condición da `T`, el `cond` devuelve `T`. Debe ser `((equal estado 'LISTO) 'ENTREGAR)`.

MÓDULO 3: El Molde Sagrado de la Recursividad (Soluciones)

Ejercicio 3.1 y 3.2 — V/F y Traza de Pila

FALSO **1.** El neutro de contar debe ser `0`, no `nil` (de lo contrario hace `(+ 1 nil)`).

VERDADERO **2.** Pasar la lista entera sin `cdr` causa recursión infinita.

FALSO **3.** Para filtrar se construye con `cons`, no con `+`.

VERDADERO **4.** El caso base debe ir primero en el `cond`.

Traza de llamada `sumar-precios '(1000 2000 1500)`:

Ida: `(+ 1000 (+ 2000 (+ 1500 0)))` → Vuelta: 1500 → 3500 → **4500**.

Ejercicio 3.3 — Código de los Tres Patrones

```
;; Contar:
(defun contar-caros (lista limite)
    (cond
        ((null lista) 0)
        ((> (car lista) limite) (+ 1 (contar-caros (cdr lista) limite)))
        (T (contar-caros (cdr lista) limite))))

;; Filtrar:
(defun filtrar-para-llevar (pedidos)
    (cond
        ((null pedidos) nil)
        ((equal (caddr (car pedidos)) 'LLEVAR)
            (cons (car pedidos) (filtrar-para-llevar (cdr pedidos))))
        (T (filtrar-para-llevar (cdr pedidos)))))

;; Universal:
(defun todas-libres-p (mesas)
    (cond
        ((null mesas) T)
        ((not (equal (car mesas) 'LIBRE)) nil)
        (T (todas-libres-p (cdr mesas)))))
```

MÓDULO 4: MAPCAR y Funciones de Orden Superior (Soluciones)

Ejercicio 4.1 — ¿Recursión o MAPCAR?

- 1. **MAPCAR:** Salida del mismo largo (transformación 1 a 1 de pedidos a precios).
- 2. **RECURSIÓN:** Es un filtro (la lista resultante tiene menos elementos).
- 3. **RECURSIÓN:** Reduce la lista a un único número escalar (promedio).
- 4. **MAPCAR:** Salida del mismo largo (a cada tiempo le corresponde un mensaje).
- 5. **MAPCAR:** Dos listas en paralelo que producen una lista de igual longitud.
- 6. **RECURSIÓN:** Búsqueda existencial con corte temprano que devuelve T/NIL.

Ejercicio 4.2 y 4.4 — Retornos y Parámetro Escalar

- `(mapcar 'cadr ... )` → **(CAFE TE SUB)**
- `(mapcar '+ ... )` → **(1200 2300 3400)**
- `(mapcar > 1500 ... )` → **(NIL NIL T T)**
- `(mapcar if caro ... )` → **(ACCESIBLE CARO ACCESIBLE CARO)**
- `(mapcar * ... )` → **(3000 6000 4000)**

> **Error del parámetro escalar:** Pasar `porcentaje` como lista a mapcar hace que Lisp falle porque `0.10` no es una lista. Se resuelve con cierre léxico: el lambda solo recibe `(precio)` y toma `porcentaje` del entorno.

MÓDULO 5: El Gimnasio Didáctico (Soluciones)

Ejercicio 5.1 — Pareo de Enunciados

1 → **E** (Contar: neutro 0, suma 1)

2 → **C** (Mensajes descriptivos: mapcar con lambda y cond)

3 → **D** (Filtro: neutro nil, combina con cons)

4 → **A** (Universal: neutro T, corta en nil)

5 → **B** (Sumar campo: neutro 0, suma acumulada)

MÓDULO 6: Simulacro Integrador Completo (Solución Verificada)

Solución Completa Lisp

```
;; Accesores
(defun obtener-grano (pedido)  (car (cadr pedido)))
(defun obtener-tostado (pedido)(cadr (cadr pedido)))
(defun obtener-precio (pedido) (cadddr pedido))
(defun obtener-tiempo (pedido) (car (cddddr pedido)))

;; Predicado y clasificador
(defun es-colombia-p (pedido)
    (equal (obtener-grano pedido) 'COLOMBIA))

(defun clasificar-espera (pedido)
    (cond
        ((≤ (obtener-tiempo pedido) 5)  (list (car pedido) 'EXPRESS))
        ((≤ (obtener-tiempo pedido) 12) (list (car pedido) 'NORMAL))
        (T                              (list (car pedido) 'DEMORADO))))

;; Lista de pedidos
(defun informe-tiempos (lista-pedidos)
    (mapcar 'clasificar-espera lista-pedidos))

(defun recaudacion-total (lista-pedidos)
    (cond
        ((null lista-pedidos) 0)
        (T (+ (obtener-precio (car lista-pedidos))
              (recaudacion-total (cdr lista-pedidos))))))

(defun pedidos-colombianos (lista-pedidos)
    (cond
        ((null lista-pedidos) nil)
        ((es-colombia-p (car lista-pedidos))
            (cons (car lista-pedidos)
                  (pedidos-colombianos (cdr lista-pedidos))))
        (T (pedidos-colombianos (cdr lista-pedidos)))))

(defun contar-demorados (lista-pedidos tiempo-limite)
    (cond
        ((null lista-pedidos) 0)
        ((> (obtener-tiempo (car lista-pedidos)) tiempo-limite)
            (+ 1 (contar-demorados (cdr lista-pedidos) tiempo-limite)))
        (T (contar-demorados (cdr lista-pedidos) tiempo-limite))))

(defun todos-express-p (lista-pedidos)
    (cond
        ((null lista-pedidos) T)
        ((> (obtener-tiempo (car lista-pedidos)) 5) nil)
        (T (todos-express-p (cdr lista-pedidos)))))

(defun aplicar-promo (lista-pedidos porcentaje)
    (mapcar (lambda (pedido)
```

```
                (* (obtener-precio pedido) (- 1 porcentaje)))
            lista-pedidos))

;; Menu interactivo
(defun menu-cafeteria ()
    (let ((pedidos) (tiempo-max))
        (print "═══ SISTEMA DE GESTION CAFETERIA EL BUEN GRANO ═══")
        (print "Ingrese la lista de pedidos: ")
        (setq pedidos (read))
        (print "Ingrese el tiempo maximo de espera aceptable (minutos): ")
        (setq tiempo-max (read))

        (if (and (consp pedidos) (numberp tiempo-max))
            (progn
                (print "INFORME DE TIEMPOS:")
                (print (informe-tiempos pedidos))
                (print "RECAUDACION TOTAL:")
                (print (recaudacion-total pedidos))
                (print "PEDIDOS COLOMBIANOS:")
                (print (pedidos-colombianos pedidos))
                (print "CANTIDAD DEMORADOS:")
                (print (contar-demorados pedidos tiempo-max))
                (print "TODOS EXPRESS?:")
                (print (todos-express-p pedidos))
                (print "PRECIOS CON 10% PROMO:")
                (print (aplicar-promo pedidos 0.10)))
            (print "ERROR: Entrada invalida."))))
```